Huzaifa Azhar



Software Engineer | Python, AI & Automation | Backend & Full-Stack Developer

Faisalabad, Pakistan | hazhar.contact@gmail.com | +92 303 8666496
LinkedIn: linkedin.com/in/huzaifa-azhar | GitHub: github.com/iPythonezta | Fiverr: fiverr.com/huzaifa_blogger

PROFESSIONAL SUMMARY

Software Engineer specialising in Python backends, AI agents and automation. Freelance developer since 2022 with 90+ completed orders and a 4.9/5 average rating (Fiverr Level 2). Builds REST APIs with Django, Django REST Framework, Flask and FastAPI; React and TypeScript frontends; LLM-powered workflows on OpenAI and Azure OpenAI; and data pipelines on PostgreSQL, MySQL and SQLite. Experience includes an industry-sponsored smart bus-stop system (InLights, NSTP), a Microsoft Imagine Cup 2026 multi-agent platform, a search engine built from first principles, and production apps deployed to Azure with CI/CD. BE Software Engineering student at NUST (2024–2028).

TECHNICAL SKILLS

Programming Languages: Python, JavaScript, TypeScript, C++, Java, SQL

Backend: Django, Django REST Framework, Flask, FastAPI, SQLAlchemy, WebSockets / SSE, REST API design

Frontend: React, Next.js, Vite, Tailwind CSS, Framer Motion, Material UI

AI / Machine Learning: OpenAI / Azure OpenAI, Multi-agent systems, RAG + ChromaDB, Prophet, HMMs (hmmlearn), scikit-learn, pandas / NumPy, Embeddings

Automation: Selenium, Scheduling & workers, PyInstaller packaging, 3rd-party API integration, Web scraping

Databases & Data: PostgreSQL, MySQL, SQLite, Turso / libSQL, msgpack / orjson, Search indexing

Cloud & DevOps: Azure, GitHub Actions CI/CD, Modal, Git

Engineering Practices: OOP, SOLID & GoF patterns, Unit & integration testing, Technical writing, Basic penetration testing

PROFESSIONAL EXPERIENCE

Freelance Programmer & Web Developer

Fiverr · Level 2 seller | Aug 2022 – Present

- Delivered 90+ orders for international clients with a 4.9 average star rating.
- Backend & APIs: Django, Django REST Framework, Flask, FastAPI
- Frontend: React / TypeScript dashboards and landing pages
- Automation: browser automation, scheduling workers, scrapers
- AI workflows: GPT-based classification, extraction and form filling
- Delivery: packaged desktop apps (PyInstaller) for non-technical users

Team Member, Microsoft Imagine Cup 2026

ZeroDevAI · team of four | 2026

- Built the tool-calling code-generation agent for a voice-to-app platform (requirements, coding and deploy agents on Azure OpenAI).

Web Development Instructor (volunteer)

App Dev League Inc | Aug 2022 – Present

- Teaches web development fundamentals to young learners as a volunteer instructor.

PROJECTS

Smart Bus Islamabad

Four-person NUST team · industry project assigned by InLights Solutions (NSTP) | 2025

Technologies: Python, Django 5, Django REST Framework, MySQL, Mapbox, React, TypeScript, Vite, Tailwind, TanStack Query, i18next

- Led backend development (13 of 14 commits) of an industry-sponsored smart bus-stop system assigned by InLights Solutions (NSTP) to a four-person NUST team.
- Built a Django REST Framework API over MySQL using a hand-written raw-SQL data layer for routes, stops, buses, display units, advertisements, advertisers and announcements.
- Implemented road-based, traffic-aware arrival-time (ETA) calculation with the Mapbox Directions API, replacing straight-line distance estimates.
- Designed content feeds for pole-mounted SMD displays combining live ETAs, severity-tagged service announcements and advertisements scheduled per display and time window.
- Secured the API with token authentication and admin/staff roles; supported GPS and SMD display simulators for end-to-end demos without hardware.

Backend (Smart-Bus-System): github.com/iPythonezta/Smart-Bus-System | Frontend (SmartBusFrontend): github.com/iPythonezta/SmartBusFrontend | Full demo on YouTube: youtu.be/5G0bPdQLY-s

ZeroDevAI

Team of four for Microsoft Imagine Cup 2026 | 2026

Technologies: Python, Flask, Socket.IO, SQLAlchemy, React 18, Vite, Tailwind, Azure OpenAI, Azure AI Speech, Azure Container Apps

- Built the code-generation agent for ZeroDevAI, a voice-to-app platform developed by a four-person team for Microsoft Imagine Cup 2026.
- Implemented a tool-calling agent loop on Azure OpenAI (read file, write file, install dependency, run build check) that iterates until the generated React app builds successfully.
- Enforced read-before-write rules and a Vite + React template so generated applications stay consistent and buildable.
- Streamed build logs and chat to the browser over Flask-SocketIO with database-backed session restore.

BigSearch

Three-person DSA course project | 2025

Technologies: Python, C++, msgpack, orjson, NumPy, pandas, TF-IDF, PageRank, Trie

- Co-built a full-text search engine over ~48,000 crawled web pages and the CORD-19 research-paper corpus with a ~1.4 million-term lexicon.
- Implemented the inverted index for web pages, with zone-weighted hit lists (title, headings, meta description, anchors, URL, body).
- Split the inverted index into barrels, converted them from JSON to msgpack and memory-mapped them, benchmarking load times to justify the format.
- Built hybrid ranking that blends keyword scores, PageRank/domain rank and TF-IDF-weighted word-embedding similarity, with a word-level LRU cache.
- Main contributor (9 of 13 commits) to BigSearchServer, a Flask REST API and React UI that serves search and autocomplete and supports incremental indexing through delta barrels.

Processing & indexing pipeline (BigSearch): github.com/iPythonezta/BigSearch | Search server & UI (BigSearchServer): github.com/iPythonezta/BigSearchServer

DraftGenius

Two-person project | 2026

Technologies: Python, Flask, hmmlearn, scikit-learn, pandas, NumPy, React 18, TypeScript, Vite, Tailwind, Recharts, Framer Motion

- Co-built an AI fantasy-cricket team optimiser for the Pakistan Super League (PSL) that generates Safe, Balanced and Explosive XIs for any fixture.
- Modelled player form with Hidden Markov Models (cold, average, hot) and a chronological Elo rating across eight T20 leagues.
- Optimised team selection with a genetic algorithm under budget, role and per-franchise constraints.
- Streamed generation-by-generation fitness to a React/TypeScript dashboard using Server-Sent Events; data pipeline uses parquet caching of Cricsheet match data.

Source on GitHub: github.com/Asim-Shoaib/FantasyPredictorServer

PakWarera

Small community team | 2026

Technologies: Python 3.12, Flask, Prophet, Turso (libSQL), Azure PostgreSQL, Azure Web Apps, Modal, Discord OAuth2, Web Push, GitHub Actions

- Built and maintain a Flask platform for a live strategy game, deployed to Azure Web Apps through a GitHub Actions CI/CD pipeline.
- Implemented per-item price forecasting with Prophet at 6, 12 and 24-hour horizons, tracking accuracy with MAPE.
- Integrated Discord OAuth2 with role-based access (guest, user, admin, president), citizenship approval flows and Web Push war alerts.
- Runs background workers (forecasting, recon scraping, donation sync, alerts) over Azure PostgreSQL, with a serverless Modal cron scraper writing to Turso (libSQL).
- Hardened data collection after an API change: cursor-based pagination, retry with exponential backoff, batched upserts and a priority refresh queue, all covered by unit tests.

Live site (Discord login required): pakwarera.azurewebsites.net

ACORD Form Filler

2026

Technologies: Python, Flask, OpenAI GPT, pypdf, pdfplumber, python-docx, SQLite, Vanilla JS

- Built an LLM pipeline that fills ACORD 125/126/140 commercial insurance PDFs (1,171 fillable fields) from unstructured client documents.
- Extracted text from TXT, DOCX and PDF sources and generated an AI client profile used as the mapping context.
- Mapped fields in batches of 150 with strict JSON output and a rule against inventing data, followed by a human review-and-edit step before download.
- Delivered a Flask web application with client management, document uploads, a four-step fill wizard and per-client form history.

FB Lead Radar

2026

Technologies: Python, FastAPI, SQLite, GPT-4o, Selenium, Vanilla JS, PyInstaller, Tailwind

- Shipped a desktop lead-generation tool that monitors Facebook groups and qualifies posts as leads using GPT-4o with user-defined screening criteria.
- Built a FastAPI backend with SQLite (WAL mode), a campaign scheduler and Selenium-based browser automation.
- Delivered instant email alerts and a lead feed with the AI's reason for each match.
- Packaged the application as a Windows executable with PyInstaller and built its marketing landing page.

X Automation Suite

2026

Technologies: Python, Flask, Flask-SocketIO, SQLAlchemy, OpenAI, Selenium, SQLite, PyInstaller

- Built a campaign-driven social media automation platform with per-profile posting intervals, prompts and keyword monitoring.
- Screened candidate posts with an LLM relevance filter before generating replies with OpenAI.
- Pushed a live, filterable activity log to the browser over Flask-SocketIO, backed by SQLAlchemy and SQLite.
- Distributed builds to non-technical users with PyInstaller.

Inventory Management Tool

2024 – 2026

Technologies: Django 5, Django REST Framework, React 18, Material UI, SQLite, Axios

- Built a point-of-sale and inventory system with a Django REST Framework backend and a React frontend for small retail stores.
- Implemented role-based access (cashier, manager, admin), barcode checkout, printable receipts, order history, stock alerts and a sales dashboard.
- Audited and refactored the codebase using SOLID principles and Strategy, Observer and Factory patterns, moving business logic into services and repositories.
- Fixed defects found in the audit, including stock not being restored when orders were cancelled.

Source on GitHub: github.com/iPythonezta/InventoryManagementTool

EDUCATION

Bachelor of Engineering, Software Engineering

National University of Sciences & Technology (NUST) | Sep 2024 – Sep 2028 (In progress · expected 2028)

Relevant coursework: Data Structures & Algorithms (CS-250), Software Design & Architecture, Web Engineering, Software Testing

- Data Structures & Algorithms (CS-250): BigSearch: lexicon, forward and inverted indexes, PageRank
- Software Design & Architecture: Inventory Tool refactor: SOLID principles, Strategy, Observer and Factory patterns

CERTIFICATIONS

- **IT Automation with Python** | Google | Professional Certificate
  Python scripting, interacting with the OS, Git and GitHub, debugging and troubleshooting, configuration management and cloud, automating real-world tasks.
- **Meta Back-End Developer** | Meta | Professional Certificate
  Python and Django, REST API design, MySQL and databases, version control, Linux and testing, finishing with a capstone back-end project.
- **Advanced React** | Meta | Course
  Advanced hooks, reusable component patterns, testing React apps and consuming APIs.
- **AI Engineering** | IBM | Professional Certificate
  Machine learning with Python and scikit-learn, deep learning frameworks, building and evaluating models.
- **Junior Cyber Security Analyst** | Hack The Box | Coursework completed
  Security-analyst fundamentals: networking, Linux and Windows basics, security monitoring and basic penetration-testing concepts.

ADDITIONAL PROJECTS

- **BigCrawl:** Python web crawler that respects robots.txt and uses SimHash for near-duplicate detection. (github.com/iPythonezta/BigCrawl)
- **Facebook Product Recommendation Bot:** FastAPI + React app that monitors Facebook groups and replies with product recommendations from a RAG agent over ChromaDB.

- **Jskibiddi:** Java-based interpreted programming language with Gen-Z keywords, built on the Lox interpreter design, with a Flask backend and web frontend. (github.com/iPythonezta/Jskibiddi)
- **Risk Profiler Web App:** Web app that estimates a person's money-laundering risk from their personal data. (github.com/iPythonezta/Risk-Profiler-Webapp)
- **AI-Journo:** AI journalist that gathers the latest news and publishes it on X (Twitter). (github.com/iPythonezta/AI-Journo)
- **X-ray Classifier:** Machine-learning model that classifies diseases from X-ray images and symptoms. (github.com/iPythonezta/xray-classifier)
- **Project Management Tool:** Full project management application built with Django and React. (github.com/iPythonezta/Project-Management-Tool)
- **GetCovidData:** Python package for retrieving COVID-19 statistics with simple commands. (github.com/iPythonezta/GetCovidData)

LANGUAGES

English (fluent), Urdu (fluent)